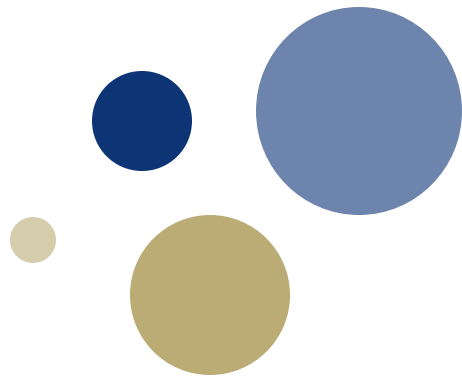




NTNU

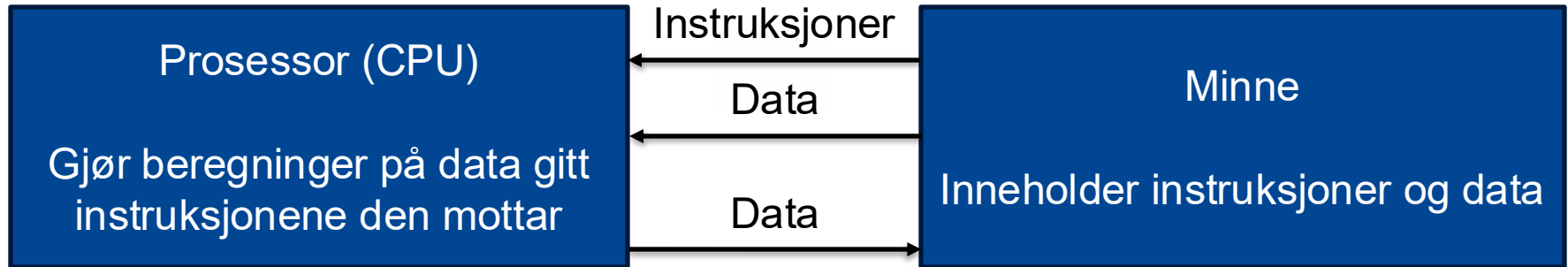
Kunnskap for en bedre verden



Forelesning 6: Flyttall og flersykelprosessor

TDT4160 Datamaskiner

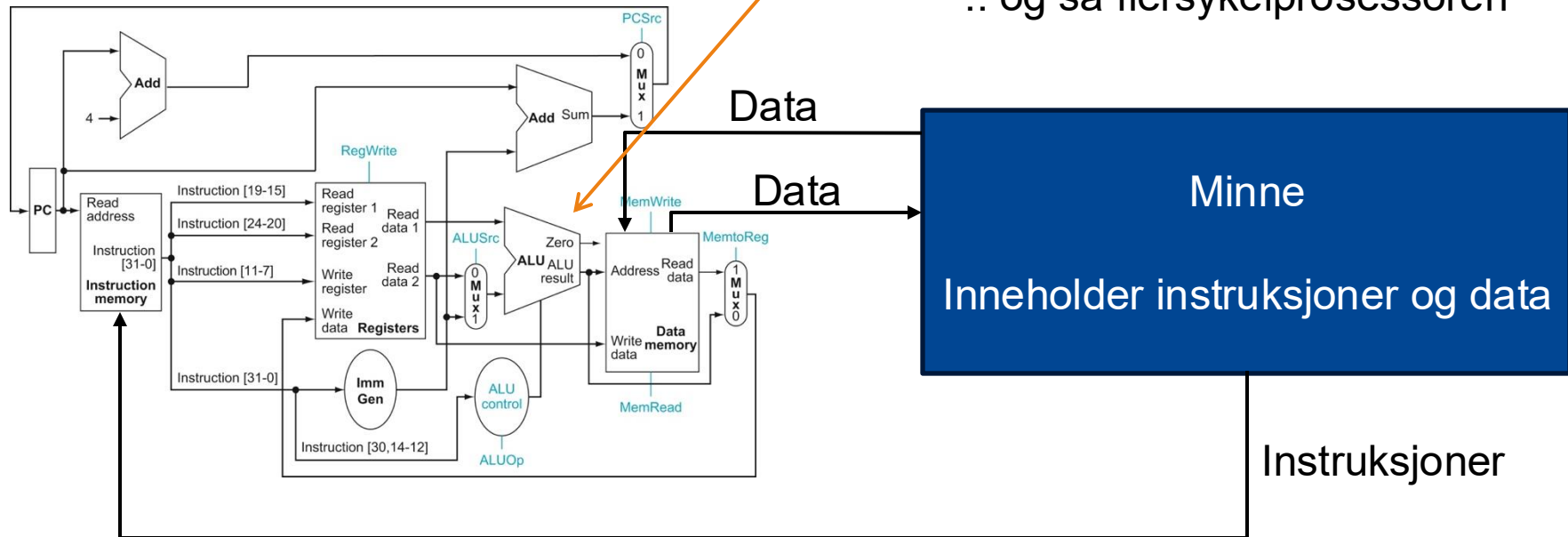
Prinsippet om lagrede program



Prinsippet om lagrede program

I dag: Først flyttall

.. og så flersykelprosessen





T3.3: Aritmetisk-logisk enhet (Kapittel 3.5)

FLYTTALL

Læringsutbytte T3.3

- Studenten skal kunne konstruere en 32-bit aritmetisk-logisk enhet (ALU) som kan gjøre addisjon og subtraksjon samt logisk AND og OR og nulldeteksjon (gjengi og forklare figurene A.5.7 og A.5.8).
- Studenten skal kunne utføre multiplikasjon og divisjon av binære tall og beskrive hvordan disse implementeres i maskinvare.
- Studenten skal kunne oppgi fordeler og ulemper med å representere tall i et «fixed-point» format.
- Studenten skal kunne motivere for hvorfor vi trenger **flyttall** og beskrive prinsippene for hvordan addisjon og multiplikasjon med flyttallsverdier utføres i maskinvare.
- Studenten skal kunne **konvertere** fra **desimaltall** til **flyttall** og motsatt.
- Studenten skal kjenne til hvordan man implementerer SIMD-instruksjoner og hva de brukes til.

Hvorfor flyttall?

- Så langt har vi fokusert på heltall, men vi må også kunne representere desimaltall
- Forrige forelesning: «Fixed point»
 - Fordel: Vi kan regne som med heltall. Eneste endring er at vi omdefinierer verdien til hver bit.
 - Ulempe: Vi må velge mellom å kunne representere store tall og små tall
- Løsning: Flyttall («floating point numbers»)
 - ... altså tallformat der komma flytter seg ved behov (flyter)

Flyttall

- Basert på vitenskapelig notasjon
 - $1123 = 1,123 \cdot 10^3$
 - $0,001123 = 1,123 \cdot 10^{-3}$
 - → Gjennom å endre på eksponenten kan vi representere både (veldig) store og (veldig) små tall

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
s		exponent								fraction																					
1 bit		8 bits								23 bits																					

$$(-1^s) \cdot \text{fraction} \cdot 2^{\text{exponent}}$$

Presisjon og rekkevidde

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
s	exponent								fraction																						
1 bit	8 bits								23 bits																						

$$(-1)^s \cdot \text{fraction} \cdot 2^{\text{exponent}}$$

- Hvor mange bit skal vi bruke på hvert felt?
 - Flere «fraction» bit øker presisjon → Vi kan representere tall mer nøyaktig
 - Flere «exponent» bit øker «range» → Vi kan representere større og mindre tall
- Flyttall er også en endelig representasjon
 - Overflyt («overflow») oppstår når tallet er for stort (i positiv eller negativ retning) til å kunne representeres med «exponent»
 - Underflyt («underflow») oppstår når tallet er for lite (i positiv eller negativ retning) til å kunne representeres med «exponent»

Eksempel: Tallinje for flyttall

7	6	5	4	3	2	1	0
Sign	Exponent				Fraction		
$(-1)^s$	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}

$$(-1)^s \cdot \text{fraction} \cdot 2^{\text{exponent}}$$

Utvalgte «Exponent» verdier:

Binær	Exponent	Desimal
0000	$2^{0+0+0+0}$	1
1000	$2^{8+0+0+0}$	256
1111	$2^{8+4+2+1}$	32 768

Utvalgte «Fraction» verdier:

Binær	Fraction	Desimal
000	0	0
001	2^{-3}	0.125
100	2^{-1}	0.5
111	$2^{-1} + 2^{-2} + 2^{-3}$	0.875

Formatet er en forenklet versjon av FP8 E4M3 (mangler bias):

<https://developer.nvidia.com/blog/floating-point-8-an-introduction-to-efficient-lower-precision-ai-training/>

Eksempel: Tallinje for flyttall

$$(-1)^s \cdot \text{fraction} \cdot 2^{\text{exponent}}$$

Utvalgte «Exponent» verdier:

Binær	Exponent	Desimal
0000	$2^{0+0+0+0}$	1
1000	$2^{8+0+0+0}$	256
1111	$2^{8+4+2+1}$	32 768

Utvalgte «Fraction» verdier:

Binær	Fraction	Desimal
000	0	0
001	2^{-3}	0.125
100	2^{-1}	0.5
111	$2^{-1} + 2^{-2} + 2^{-3}$	0.875

Flyttallstandarden IEEE 754

- De mest vanlige formatene i IEEE 754 er:
 - 32-bit «single precision»
 - 64-bit «double precision»
- Bias er 127 (1023) for enkel (dobbel) presisjon
 - Forenkler sortering fordi større eksponent alltid betyr større tall
 - Øker presisjon på bekostning av rekkevidde («range»)
- Det er fort gjort å regne feil med flyttall
 - $(A + B) + C$ er for eksempel ikke nødvendigvis lik $A + (B + C)$

Eksempel!

Single precision format

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
s		exponent								fraction																					
1 bit		8 bits								23 bits																					

$$(-1)^s \cdot \underbrace{(1 + \text{Fraction})}_{\text{«Significand»}} \cdot 2^{(\text{Exponent} - \text{Bias})}$$

Single precision		Double precision		Object represented
Exponent	Fraction	Exponent	Fraction	
0	0	0	0	0
0	Nonzero	0	Nonzero	± denormalized number
1–254	Anything	1–2046	Anything	± floating-point number
255	0	2047	0	± infinity
255	Nonzero	2047	Nonzero	NaN (Not a Number)

Eksempel: Flyttall

Single precision format

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
s	exponent								fraction																						
1 bit	8 bits								23 bits																						

$$(-1)^s \cdot (1 + \text{Fraction}) \cdot 2^{(\text{Exponent} - 127)}$$

Konverter til desimal:

Fortegn	«Exponent»	«Fraction»
1	1000 0000	1000 0000 ...

Eksempel: Flyttall

Single precision format

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
s	exponent								fraction																						
1 bit	8 bits								23 bits																						

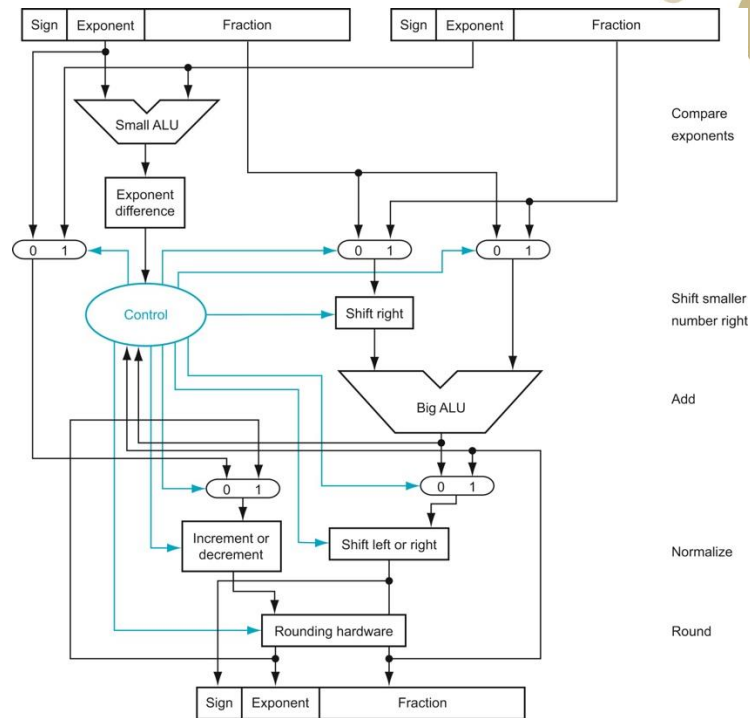
$$(-1)^s \cdot (1 + \text{Fraction}) \cdot 2^{(\text{Exponent} - 127)}$$

Representer 18,5 som et single-precision flyttall.



Maskinvare for flyttallsaddisjon

- Maskinvare for flyttallsaddisjon er komplisert men essensen er:
 - Regn ut forskjellen på eksponentene
 - Skift den miste operanden mot høyre for å få komma på samme sted
 - Legg sammen operandene
 - Skift resultatet slik at «Fraction» kommer på formen 1.xxx (normalisering)
- I høytytelsesprosessorer:
 - Flyttallsaddisjon og flyttallsmultiplikasjon tar ~4 klokkesykler
 - Flyttallsdivisjon tar fortsatt noen titalls (>20) klokkesykler



Eksempel: $0.28125 + 3.0 = ?$

$$(-1)^0 \cdot (1+0,125) \cdot 2^{(125-127)}$$

$$(-1)^0 \cdot (1+0,5) \cdot 2^{(128-127)}$$

0	0111 1101	0010 0000 ...	$+$	0	1000 0000	1000 0000 ...	$=$
	$1,125 \cdot 2^{-2}$				$1,5 \cdot 2^1$		

1 og 2: Regn ut forskjellen på eksponentene og skift den minste «significand» til potensene blir like

2^{-2}	1,0010 0000	$1,125 \cdot 2^{-2} = 0.28125$
2^{-1}	0,1001 0000	$0,5625 \cdot 2^{-1} = 0.28125$
2^0	0,0100 1000	$0,28125 \cdot 2^0 = 0.28125$
2^1	0,0010 0100	$0,140625 \cdot 2^1 = 0.28125$

3: Legg sammen «significand» verdiene (altså $1 + \text{«fraction»}$)

$$0,0010 0100 + 1,100 0000 = 1,1010 0100$$

$$1 + \frac{1}{2} + \frac{1}{8} + \frac{1}{64} = 1,640625$$

$$1 + 0,5 + 0,125 + 0,015625 = 1,640625$$

4: Normaliser (men «significand» er allerede normalisert)

$$(1+0,640625) \cdot 2^{128-127} = 3,28125$$

0	0111 1101	0010 0000 ...	$+$	0	1000 0000	1000 0000 ...	$=$	0	1000 0000	1010 0100 ...
---	-----------	---------------	-----	---	-----------	---------------	-----	---	-----------	---------------

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
s		exponent										fraction																			
1 bit		8 bits										23 bits																			

$$(-1)^s \cdot (1 + \text{Fraction}) \cdot 2^{(\text{Exponent} - 127)}$$

Eksempel: $0.28125 \times 3.0 = ?$

$$(-1)^0 \cdot (1+0,125) \cdot 2^{(125-127)}$$

$$(-1)^0 \cdot (1+0,5) \cdot 2^{(128-127)}$$

0	0111 1101	0010 0000 ...
---	-----------	---------------

×

0	1000 0000	1000 0000 ...
---	-----------	---------------

=

$$1,125 \cdot 2^{-2} \cdot 1,5 \cdot 2^1$$

$$= 1,125 \cdot 1,5 \cdot 2^{-2+1} = 0,84375$$

1: Legg sammen eksponentene og korreger for bias

Legg sammen eksponentene: $(-2 + 127) + (1 + 127) = -1 + 127 + 127 = 253$

$$0111\ 1101 + 1000\ 0000 = 1111\ 1101$$

Korriger for bias: $253 - 127 = 126 = -1 + 127$

$$1111\ 1101 - 0111\ 1111 = 1111\ 1110$$

2: Multipliser «significand» verdiene (fortsatt $1 + \text{«fraction»}$):

$$1,125 \cdot 1,5 = 1,6875$$

$$1 + \frac{1}{2} + \frac{1}{8} + \frac{1}{16} = 1,6875$$

$$1,0010\ 0000 \times 1,1000\ 0000 = 1,1011\ 0000$$

$$(1+0,5 + 0,125 + 0,0625) = 1,6875$$

3: Normaliser (men «significand» er allerede normalisert)

$$(1+0,6875) \cdot 2^{126-127} = 0,84375$$

0	0111 1101	0010 0000 ...
---	-----------	---------------

×

0	1000 0000	1000 0000 ...
---	-----------	---------------

=

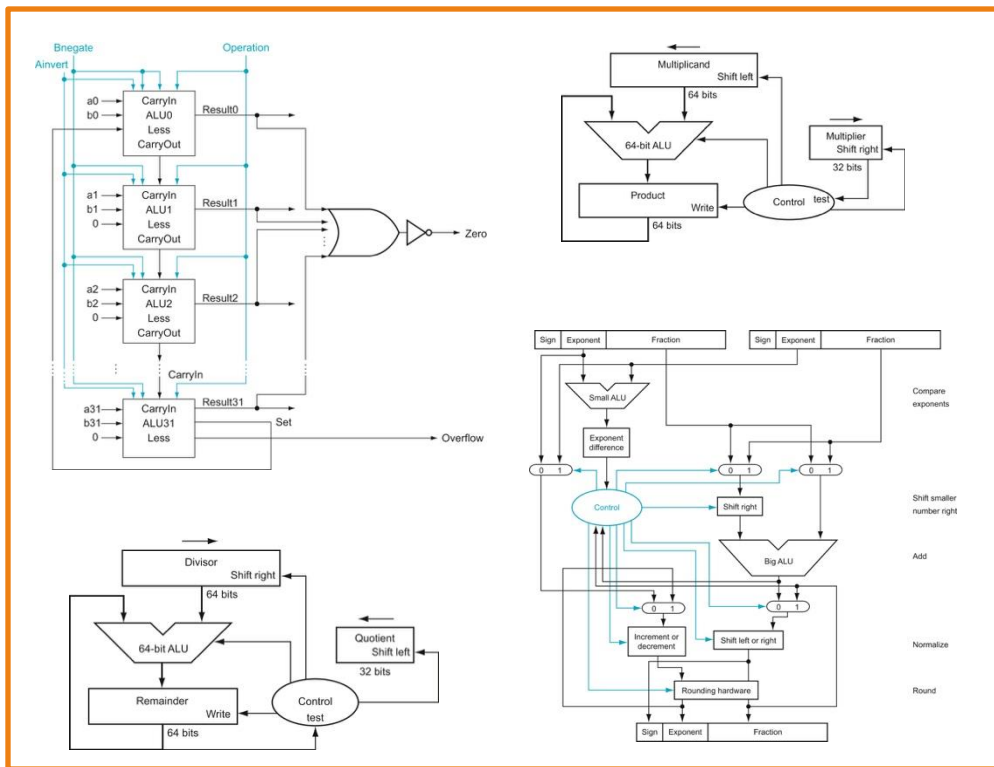
0	1111 1110	1011 0000 ...
---	-----------	---------------

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
s		exponent										fraction																			
1 bit		8 bits										23 bits																			

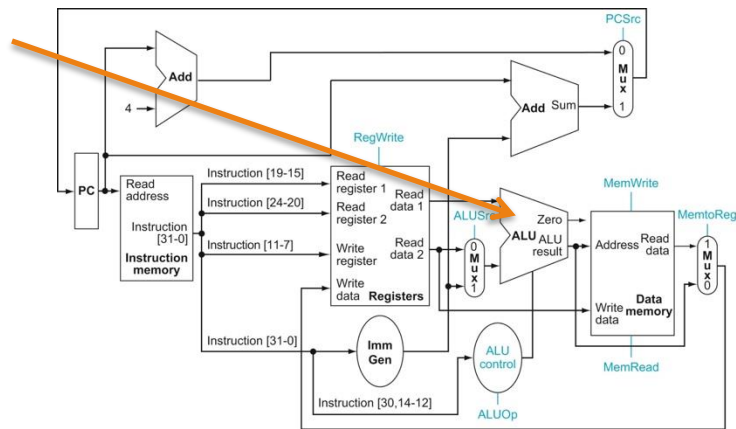
$$(-1)^s \cdot (1 + \text{Fraction}) \cdot 2^{(\text{Exponent} - 127)}$$

Oppsummering: Aritmetisk-logisk enhet

Abstraksjon!



Vi lager maskinvareenheter for alle aritmetisk-logiske operasjoner datamaskinen skal støtte ... og skruer på den maskinvare-enheten som en instruksjon trenger ved hjelp av kontrollsignaler





Tema 4.2: Sekvensiell logikk (Kapittel A.8)

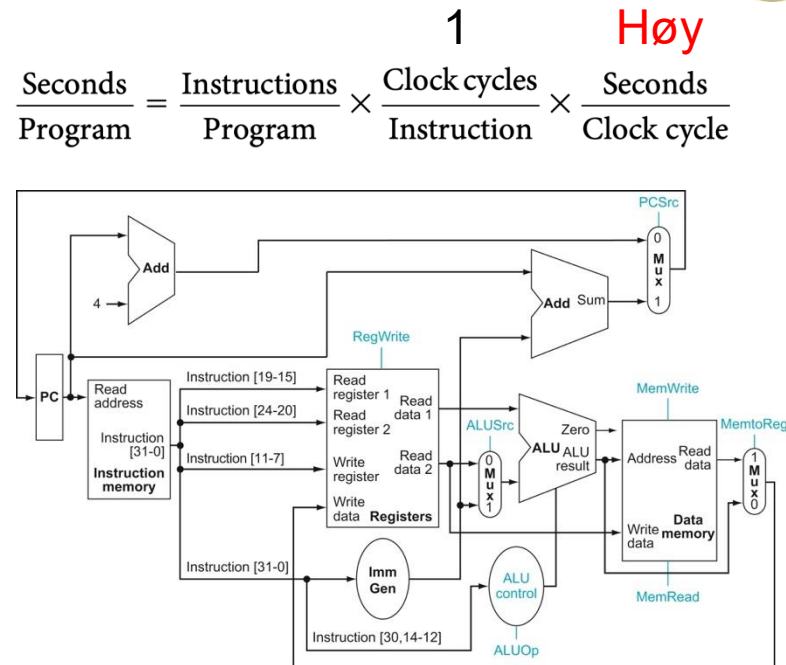
KRITISK STI

Læringsutbytte T4.2

- Studenten skal kunne beskrive oppførselen til komponentene SR-lås og D-vippe.
- Studenten skal kunne konstruere registre og registerfiler med kombinatoriske og sekvensielle logiske komponenter (kunne gjengi og forklare figur A.8.8 og figur A.8.9).
- Studenten skal kunne forklare hva en tilstandsmaskin er og beskrive hvordan den implementeres i maskinvare (kunne gjengi og forklare figur A.10.3).
- Studenten skal kunne forklare funksjonen til klokkesignalet i en datamaskin og begrepet «kritisk sti».

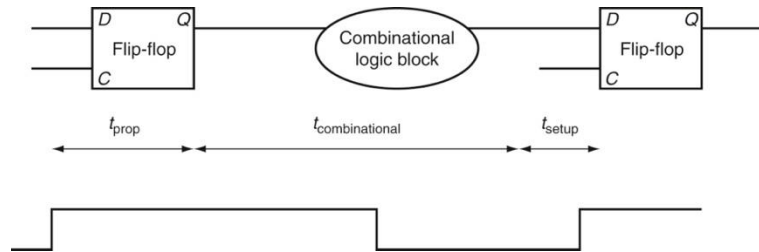
Enkeltskykelprosessoren vår har noen utfordringer...

- Kritisk sti er lang
 - ... og dette gjør at klokkefrekvensen (og dermed ytelsen) blir (relativt) lav
- Vi må ha flere adderere og minner fordi vi gjør alt i samme klokkesykel
 - Dette har en arealkostnad
 - Hvis vi bruker mer areal, vil vi gjerne også ha høyere ytelse
- Alle instruksjoner tar like lang tid
 - ... og det er ikke akkurat å gjøre det vanlige tilfellet raskt



Kritisk sti og klokkefrekvens

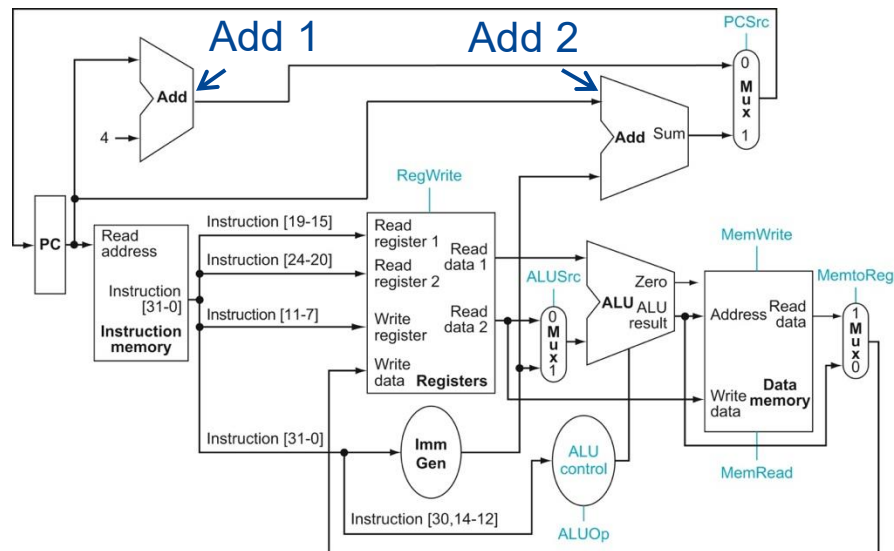
- Registeres lages med D-vipper
 - D-vipper krever at signalet er stabilt rundt klokkeflanken
 - Kalles «setup» og «hold» tid
- Det tar også tid for signalet å propagere gjennom
 - Selve D-vippen
 - Kombinatorisk logikk mellom D-vippene
- Vi må passe på at signalet rekker å propagere gjennom den kombinatoriske logikken i tide
 - Den lengste signalveien kalles kritisk sti («critical path»)
 - Forsinkelsen på kritisk sti begrenser klokkefrekvensen



Eksempel: Finne kritisk sti

Enhet	Latens (ps)
Instruction memory	200
Registers (lese)	100
Registers (skrive)	100
ALU	200
Data memory	200
Add 1	50
Add 2	200

Alle andre latenser er så små at vi kan se bort fra dem.

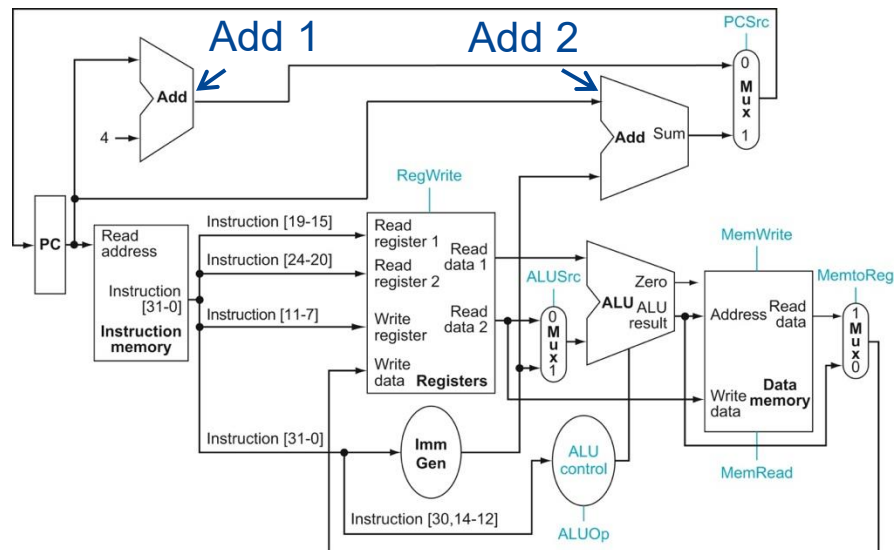


R-type:

Eksempel: Finne kritisk sti

Enhet	Latens (ps)
Instruction memory	200
Registers (lese)	100
Registers (skrive)	100
ALU	200
Data memory	200
Add 1	50
Add 2	200

Alle andre latenser er så små at vi kan se bort fra dem.

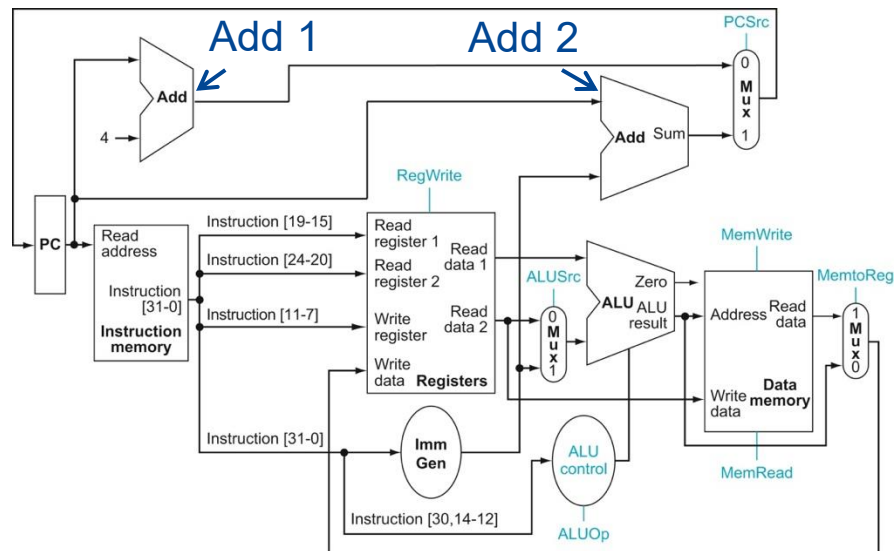


Load:

Eksempel: Finne kritisk sti

Enhet	Latens (ps)
Instruction memory	200
Registers (lese)	100
Registers (skrive)	100
ALU	200
Data memory	200
Add 1	50
Add 2	200

Alle andre latenser er så små at vi kan se bort fra dem.

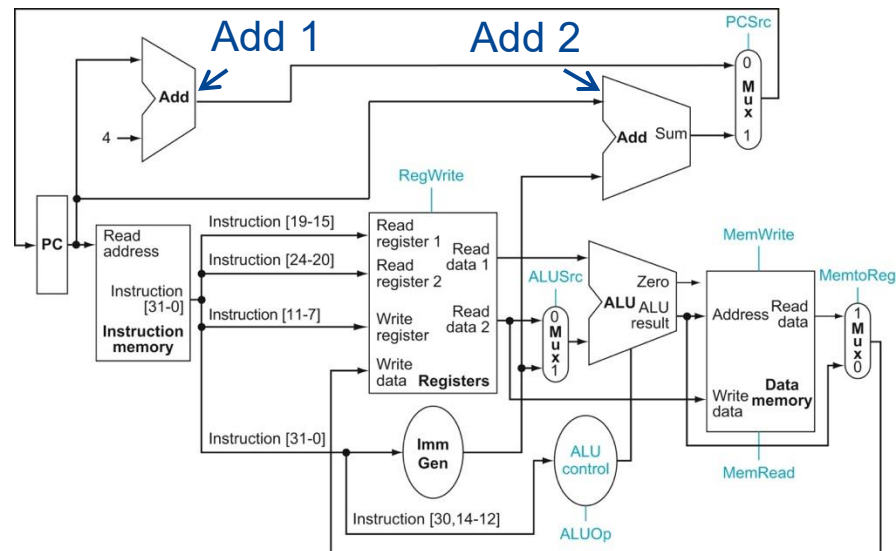


Store:

Eksempel: Finne kritisk sti

Enhet	Latens (ps)
Instruction memory	200
Registers (lese)	100
Registers (skrive)	100
ALU	200
Data memory	200
Add 1	50
Add 2	200

Alle andre latenser er så små at vi kan se bort fra dem.

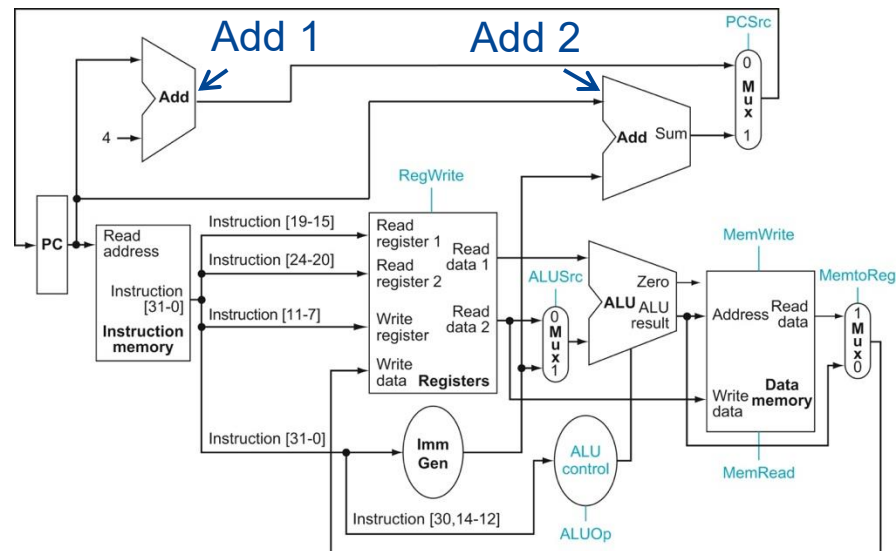


Branch equal:

Eksempel: Finne kritisk sti

Enhet	Latens (ps)
Instruction memory	200
Registers (lese)	100
Registers (skrive)	100
ALU	200
Data memory	200
Add 1	50
Add 2	200

Alle andre latenser er så små at vi kan se bort fra dem.



Instruksjonstype	Latens (ps)
R-type	600
Load	800
Store	700
Branch equal	500



Maksimal klokkefrekvens = $1/(\text{Latens på kritisk sti} + \text{slakk})$

Anta at vi trenger 200 ps slakk

$$f_{maks} = \frac{1}{\underbrace{(800 + 200)}_{\text{Klokkeperiode}} \cdot 10^{-12}} = \frac{1}{10^{-9}} = 10^9 \text{ Hz} = 1 \text{ GHz}$$



T4.1: Flersykelprosessor (Kapittel 4.5)

FLERSYKELPROSESSORER

Læringsutbytte T4.1

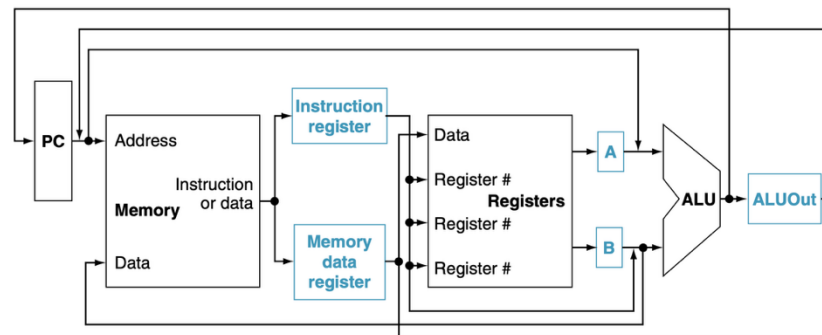
- Studenten skal kunne forklare **mikroarkitekturen** til en **flersykelprossessor** (kunne gjengi og forklare figur e.4.5.4).
- Studenten skal kunne forklare hvorfor **kontrollenheten** til flersykelprossessoren blir en **tilstandsmaskin**.
- Studenten skal kunne beskrive sammenhengen mellom **kontrollordet** til flersykelprossessoren og tilstanden i tilstandsmaskinen.
- Studenten skal kunne beskrive **fordeler og ulemper** ved **flersykelprossessoren** sammenliknet med **enkeltsykelprossessoren**.

Flersykelpro세서

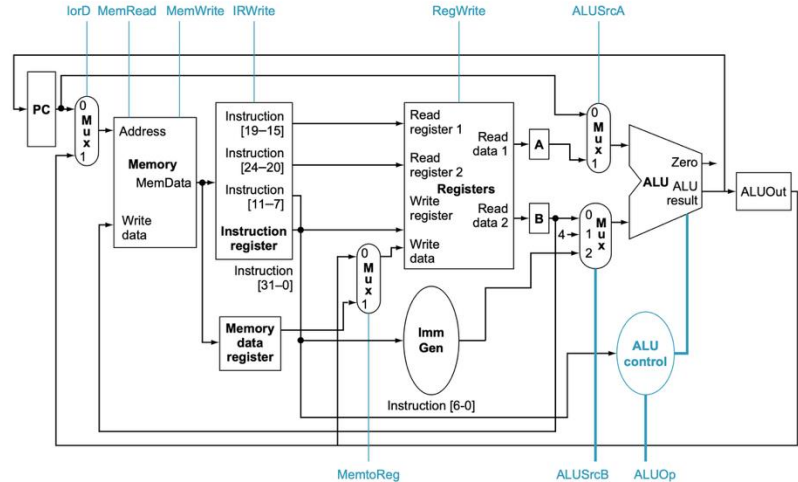
- Flersykelprosessoren bruker flere klokkesykler på en instruksjon
 - Klokkefrekvensen kan gå opp sammenliknet med enkeltsykelprossesoren fordi vi gjør mindre arbeid i en klokkesykel
 - Vi kan forenkle datastien fordi vi kan gjenbruke enheter mellom klokkesykler
- Vi trenger noen ekstra registre for å lagre verdier mellom klokkesykler
- Utfordring: Kontrollenheten blir en tilstandsmaskin

$$\frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

Høyere Lavere



-
- The diagram illustrates the MIPS R2000 processor architecture. Key components and their interconnections are as follows:
- PC (Program Counter):** Provides the address to the Memory and the next instruction to the Instruction register.
 - Memory:** Receives an address from the PC and provides MemData to the Instruction register and Memory data register. It also receives write data from the Memory data register.
 - Instruction register:** Receives instructions from the PC and provides fields to other components:
 - Instruction [19-15] to Read register 1.
 - Instruction [24-20] to Read register 2.
 - Instruction [11-7] to the Write register.
 - Instruction [31-0] to the Memory data register.
 - Instruction [6-0] to the Imm Gen.
 - Memory data register:** Receives data from the Instruction register and provides it to the Imm Gen.
 - Imm Gen (Immediate Generator):** Generates a 16-bit immediate value from the instruction and provides it to the ALU.
 - Registers:**
 - Read register 1:** Receives data from the Instruction register and provides it to Mux 1.
 - Read register 2:** Receives data from the Instruction register and provides it to Mux 2.
 - Write register:** Receives data from the Instruction register and provides it to the Register File.
 - Register File:** Receives data from the Write register and provides it to the ALU.
 - Mux 1 (Multiplexer 1):** Selects between the output of Read register 1 and the output of the ALU (Zero flag) based on the ALU control signal.
 - Mux 2 (Multiplexer 2):** Selects between the output of Read register 2 and the output of the Imm Gen based on the ALU control signal.
 - ALU (Arithmetic Logic Unit):** Performs operations on the outputs of Mux 1 and Mux 2 based on the ALU control signal. It also provides the Zero flag to Mux 1.
 - ALU control:** Receives the ALU control signal from the ALU and provides it to the ALU.
 - ALUOut:** Receives the output of the ALU and provides it to the PC.



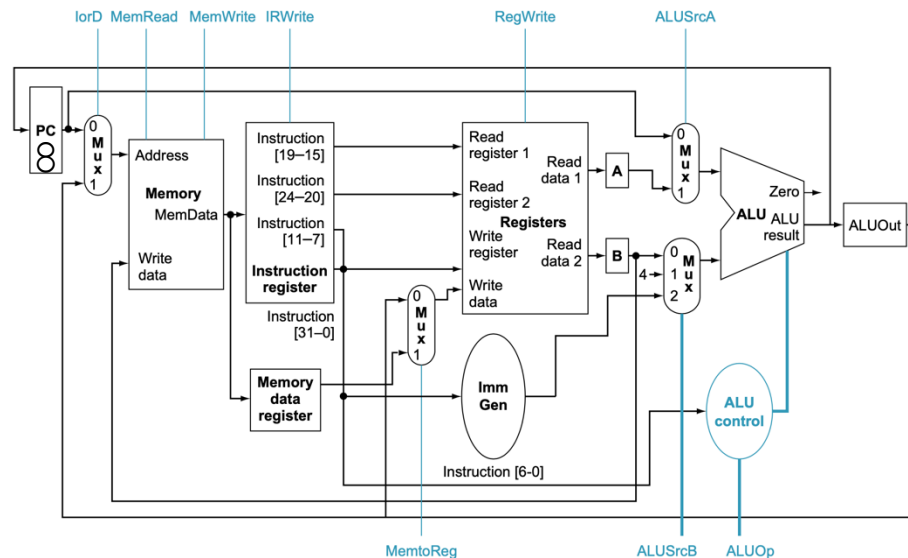


x1	17
x2	13
x3	
x4	
x5	

Instruksjonsadresse: 8

Flersykeleksempel: Add

Klokke:



Registerverdier

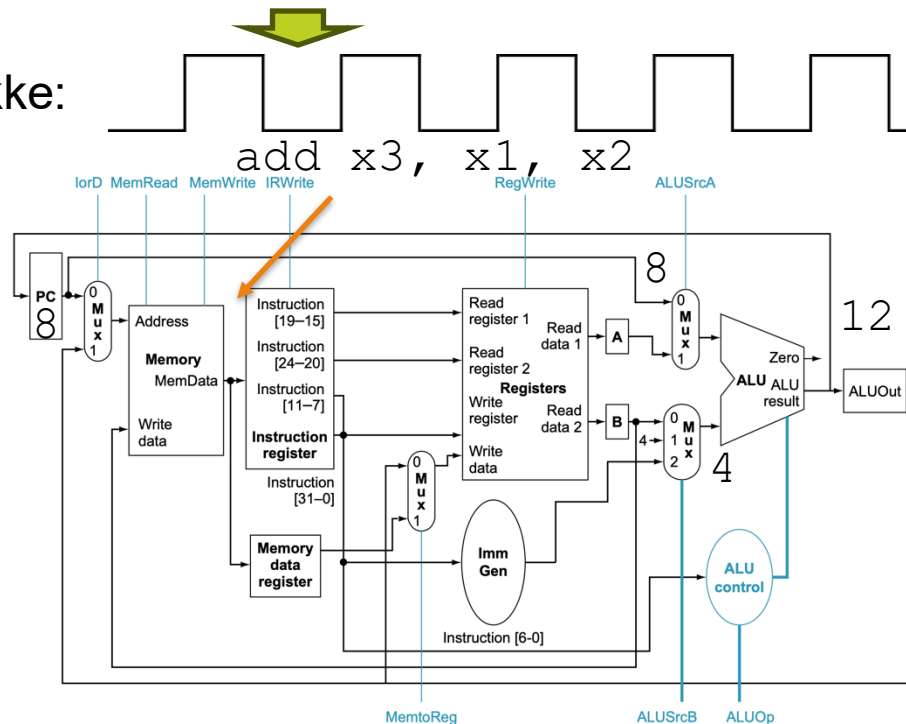
x1	17
x2	13
x3	
x4	
x5	

Instruksjon: add x3, x1, x2

Instruksjonsadresse: 8

Flersykeleksempel: Add

Klokke:



Registerverdier

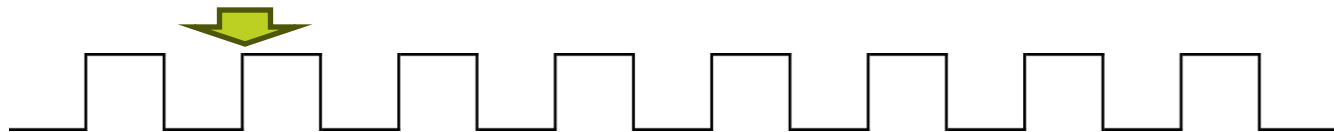
x1	17
x2	13
x3	
x4	
x5	

Instruksjon: `add x3, x1, x2`

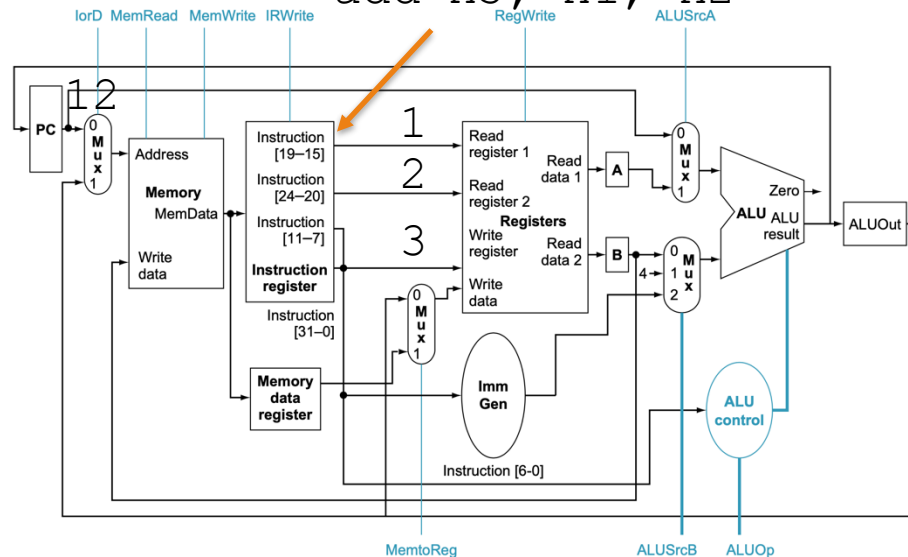
Instruksjonsadresse: 8

Flersykeleksempel: Add

Klokke:



add x3, x1, x2



Registerverdier

x1	17
x2	13
x3	
x4	
x5	

Instruksjon: add x3, x1, x2

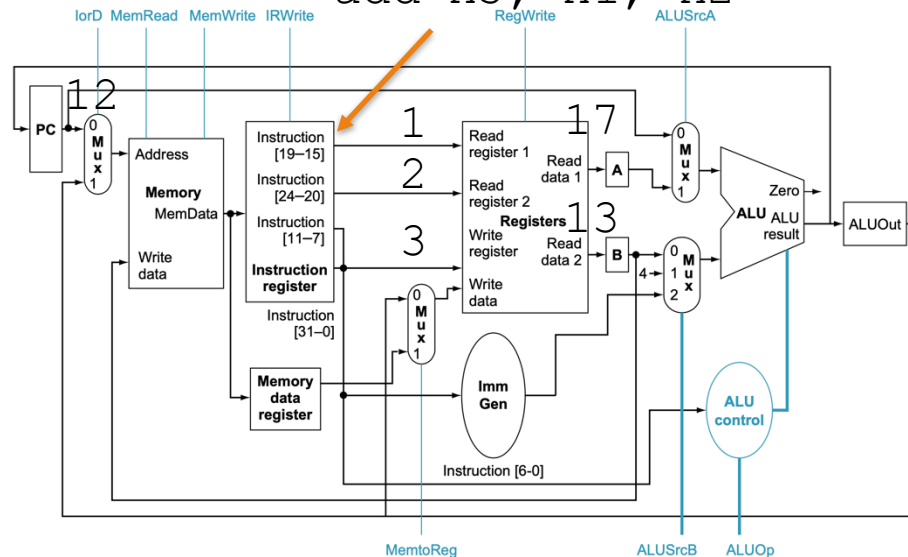
Instruksjonsadresse: 8

Flersykeleksempel: Add

Klokke:



add x3, x1, x2



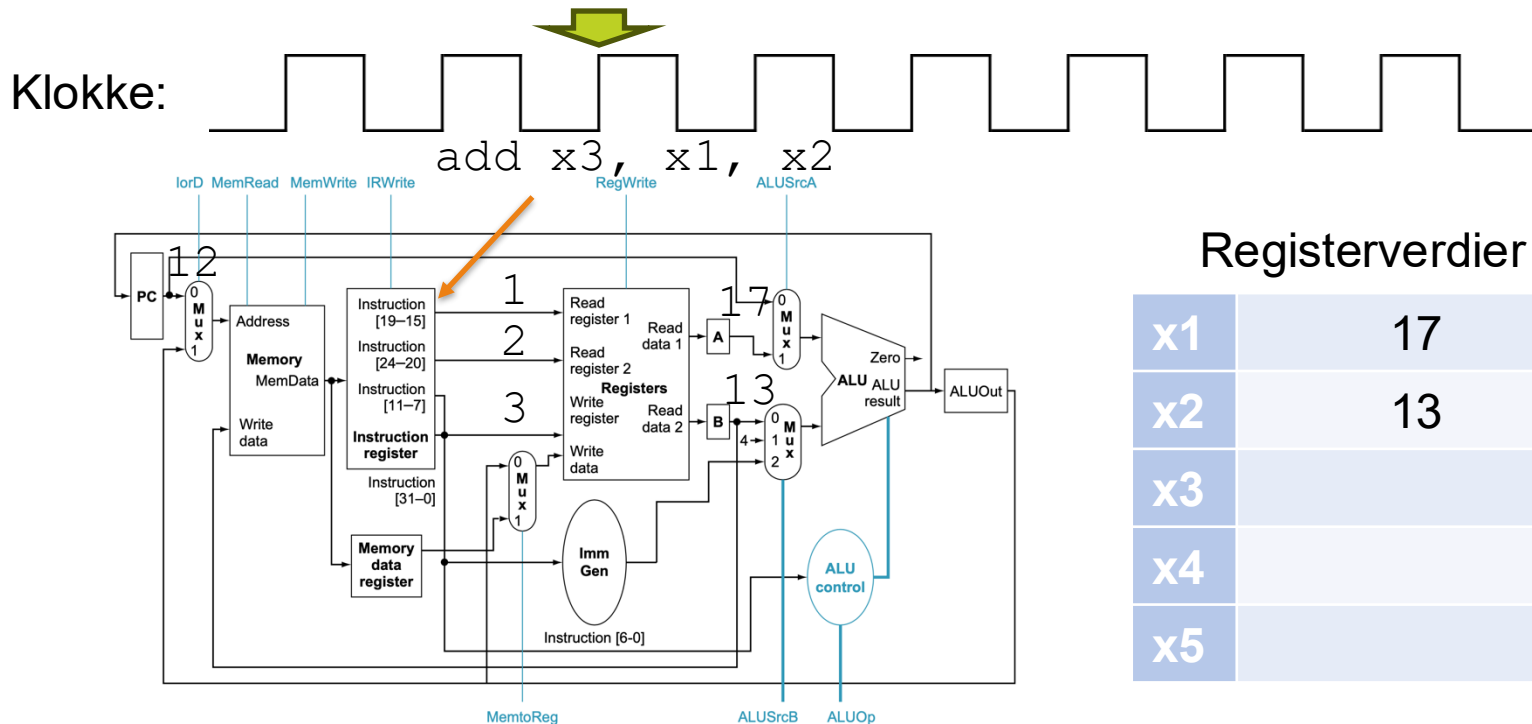
Registerverdier

x1	17
x2	13
x3	
x4	
x5	

Instruksjon: add x3, x1, x2

Instruksjonsadresse: 8

Flersykeleksempel: Add



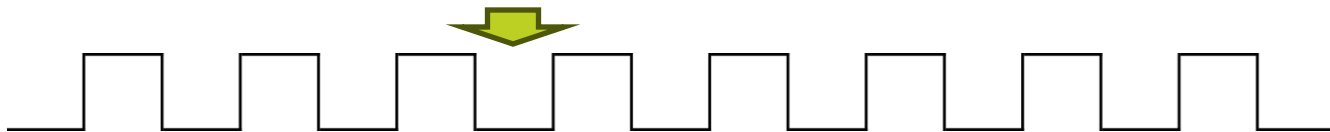
Registerverdier	
x1	17
x2	13
x3	
x4	
x5	

Instruksjon: add x3, x1, x2

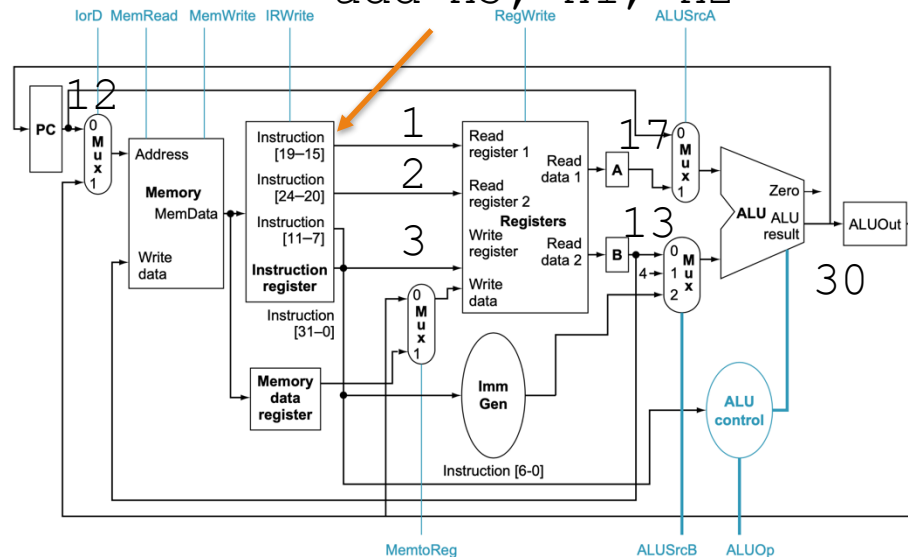
Instruksjonsadresse: 8

Flersykeleksempel: Add

Klokke:



add x3, x1, x2



Registerverdier

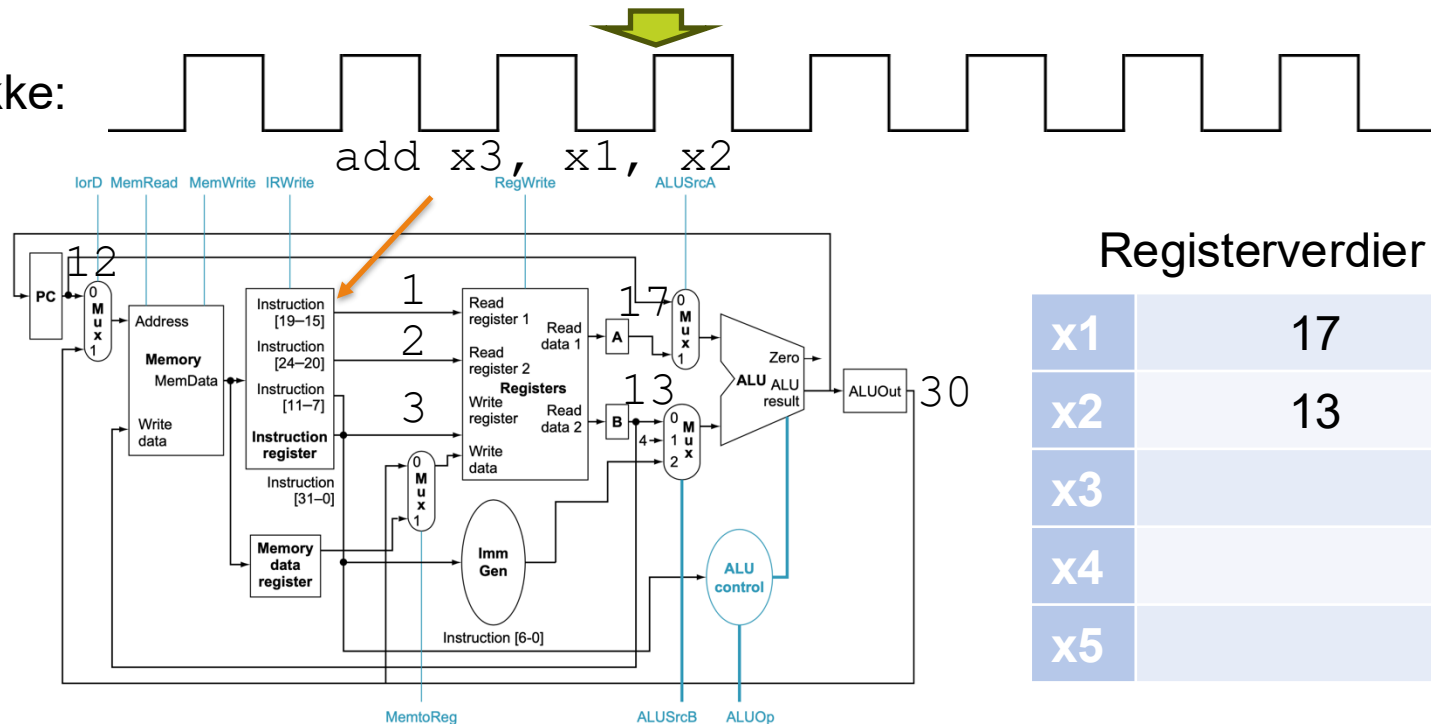
x1	17
x2	13
x3	
x4	
x5	

Instruksjon: add x3, x1, x2

Instruksjonsadresse: 8

Flersykeleksempel: Add

Klokke:



Registerverdier

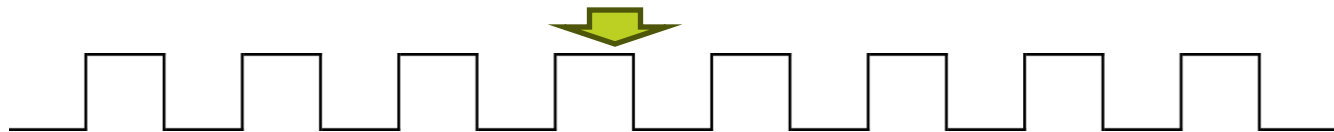
x1	17
x2	13
x3	
x4	
x5	

Instruksjon: `add x3, x1, x2`

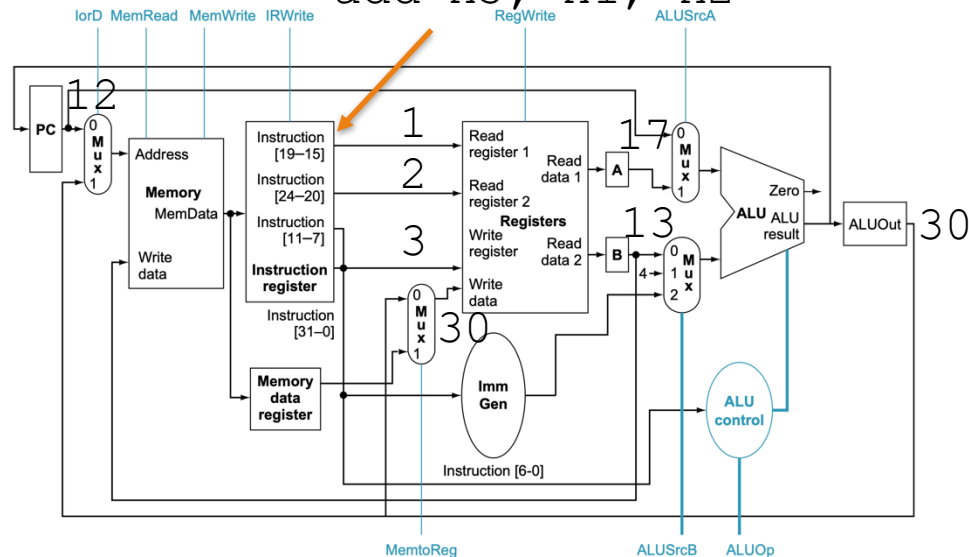
Instruksjonsadresse: 8

Flersykeleksempel: Add

Klokke:



add x3, x1, x2



Registerverdier

x1	17
x2	13
x3	
x4	
x5	

Instruksjon: add x3, x1, x2

Instruksjonsadresse: 8

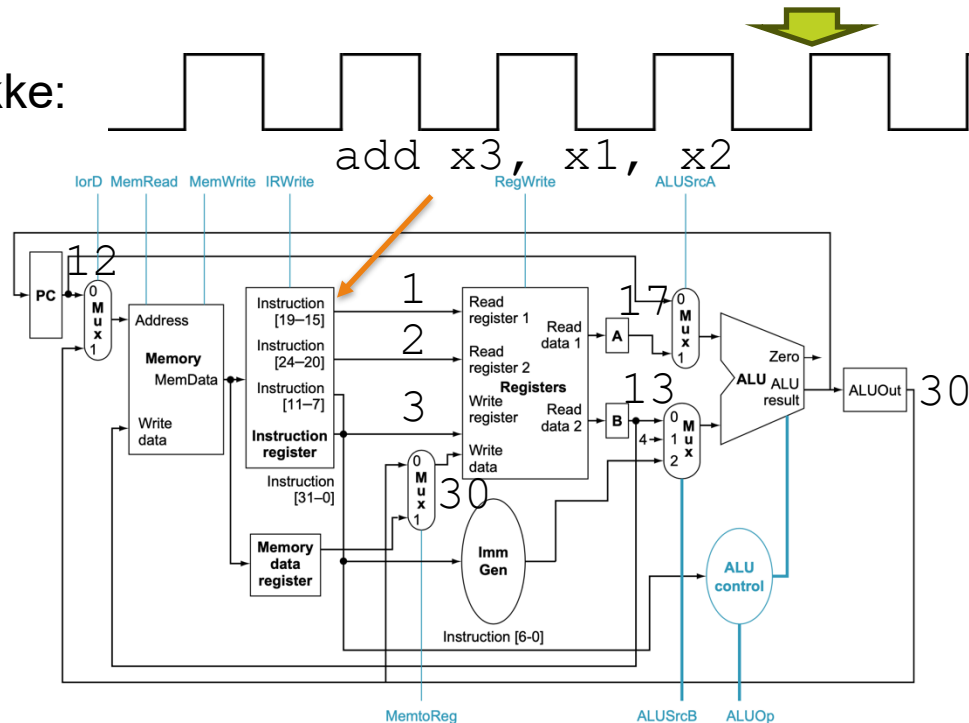


x1	17
x2	13
x3	
x4	
x5	

Instruksjonsadresse: 8

Flersykeleksempel: Add

Klokke:



Registerverdier

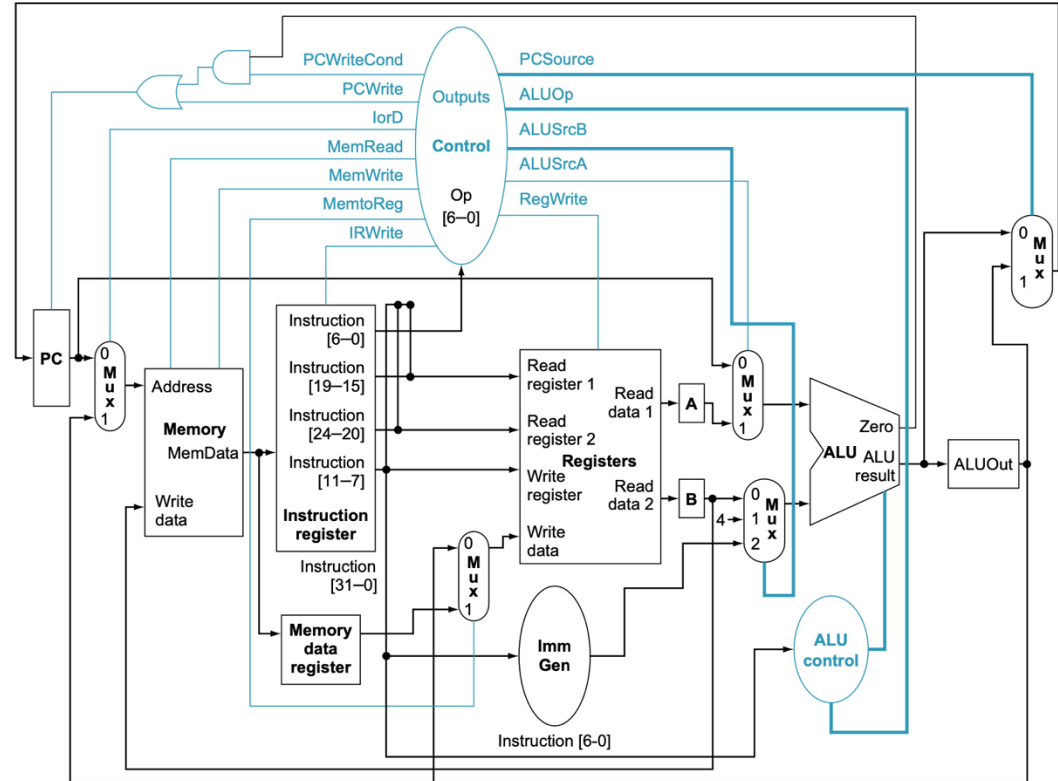
x1	17
x2	13
x3	30
x4	
x5	

Instruksjon: add x3, x1, x2

Instruksjonsadresse: 8

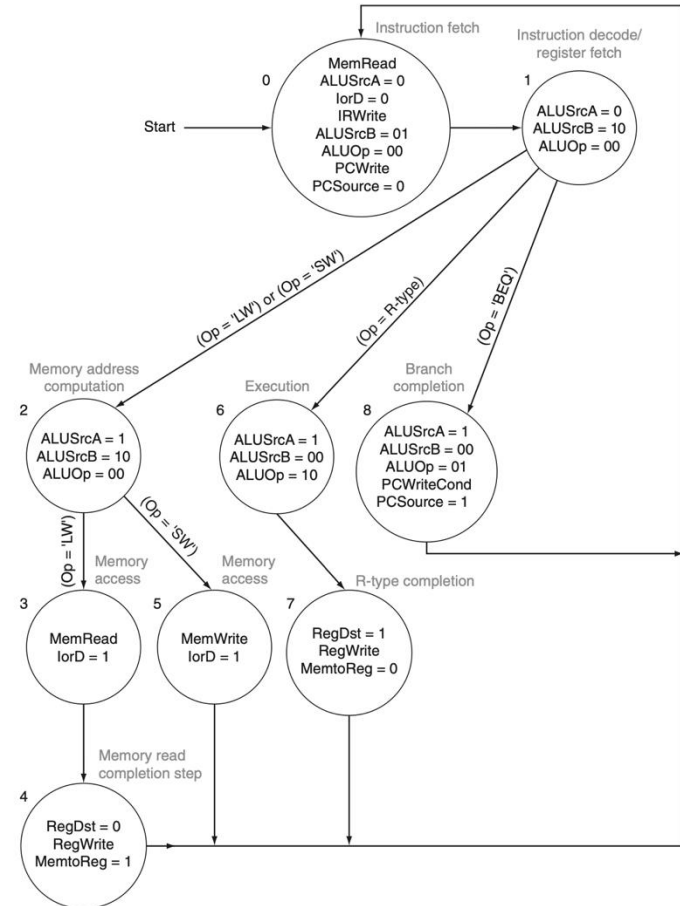
Datasti og kontrollenhet

Step name	Action for R-type instructions	Action for memory reference instructions	Action for branches
Instruction fetch	IR $\leftarrow$ Memory[PC] PC $\leftarrow$ PC + 4		
Instruction decode/register fetch	A $\leftarrow$ Reg [IR[19:15]] B $\leftarrow$ Reg [IR[24:20]] ALUOut $\leftarrow$ PC + immediate		
Execution, address computation, branch/jump completion	ALUOut $\leftarrow$ A op B	ALUOut $\leftarrow$ A + immediate	if (A == B) PC $\leftarrow$ ALUOut
Memory access or R-type completion	Reg [IR[11:7]] $\leftarrow$ ALUOut	Load: MDR $\leftarrow$ Memory[ALUOut] or Store: Memory [ALUOut] $\leftarrow$ B	
Memory read completion		Load: Reg[IR[11:7]] $\leftarrow$ MDR	



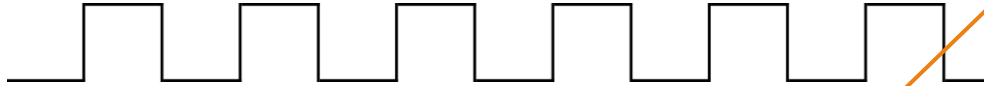
Tilstandsmaskinen for kontrollenheten

- Hver node angir kontrollsignalene i en klokkesykel
- Kantene angir hvilken tilstand vi skal gå til i neste klokkesykel
 - Hvis det er flere kanter ut av en node, angir signalene på kanten hvilken vi skal følge
 - Hvis det bare er en kant følger vi den



Tilstandsmaskin for Add

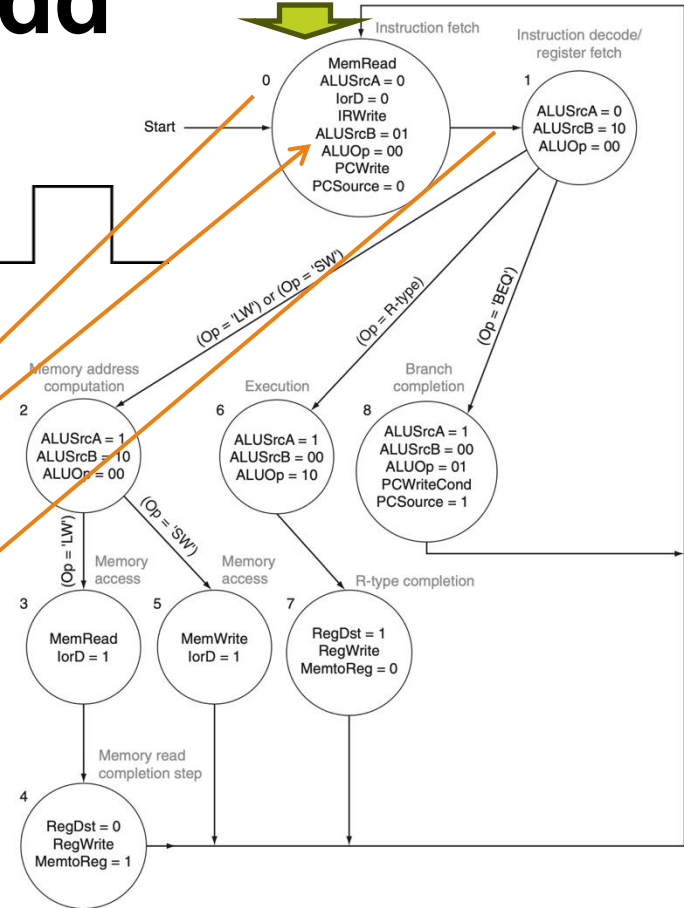
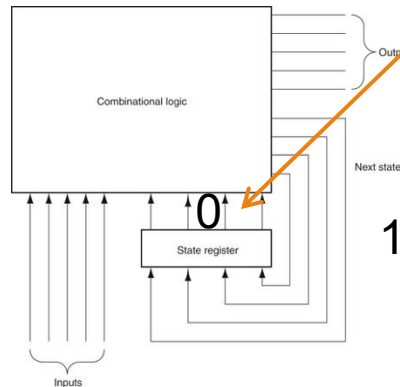
Klokke:



- Instruksjon:

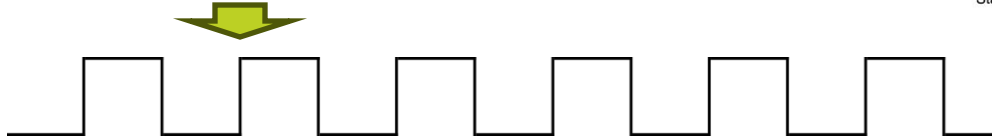
– add x3, x1, x2

Op = R-type

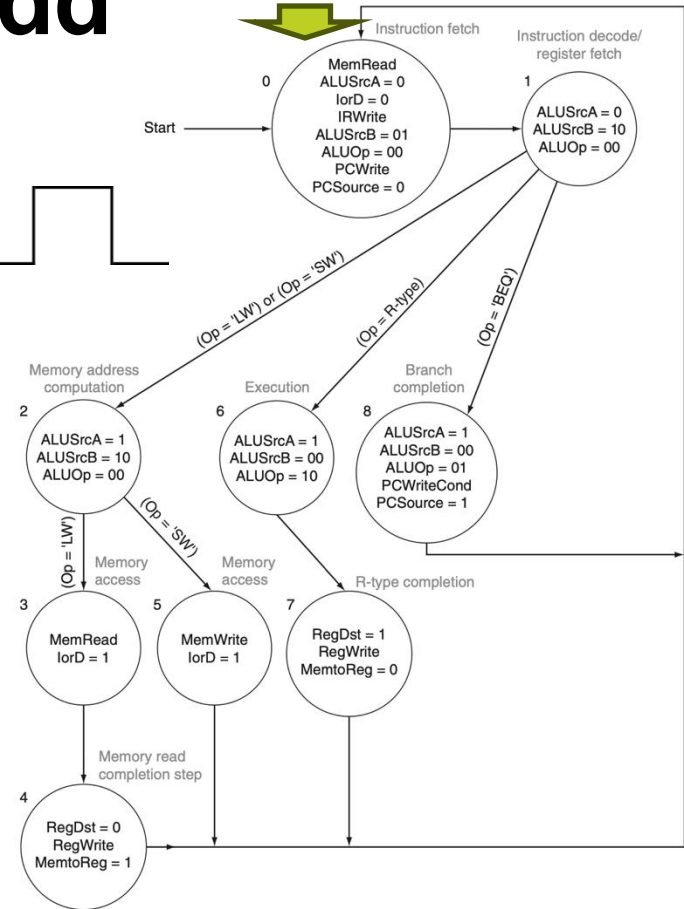
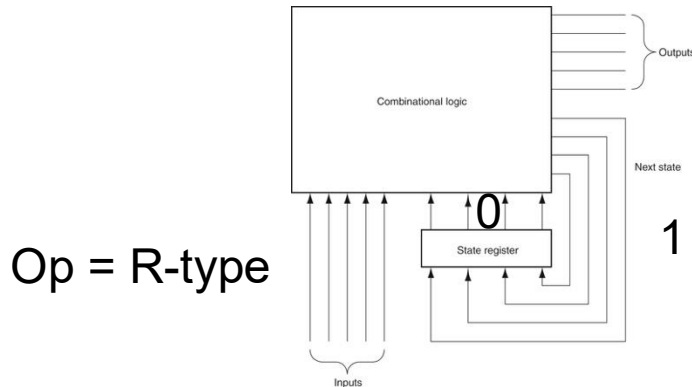


Tilstandsmaskin for Add

Klokke:

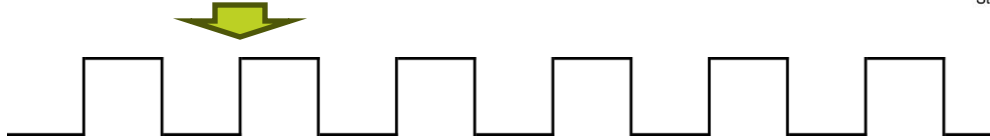


- Instruksjon:
 - add x3, x1, x2



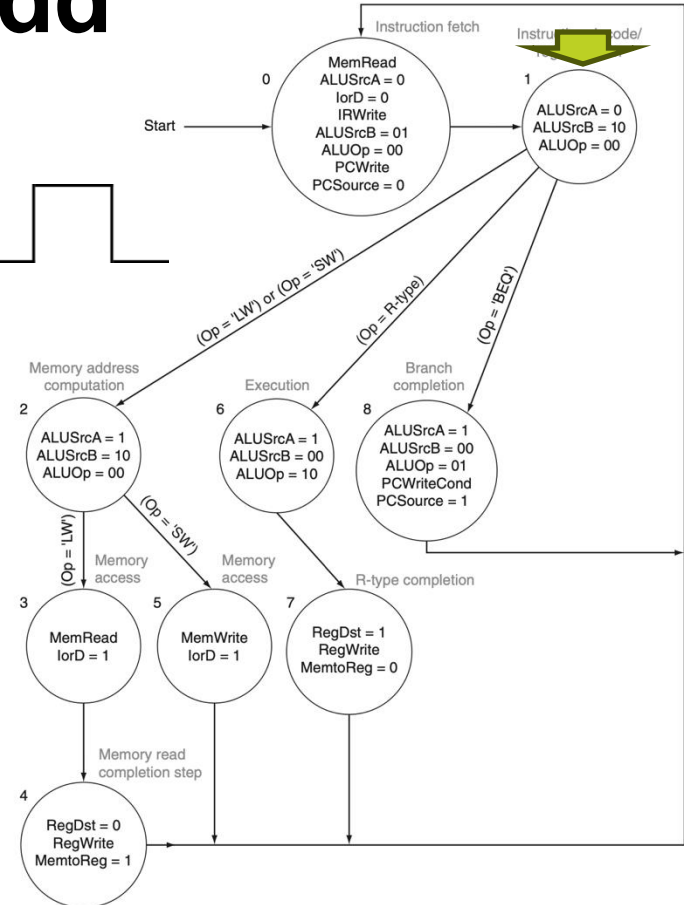
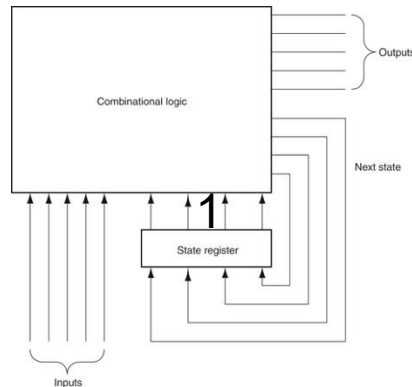
Tilstandsmaskin for Add

Klokke:



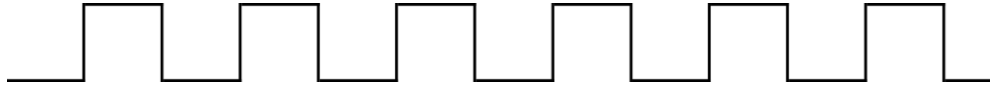
- Instruksjon:
 - add x3, x1, x2

Op = R-type



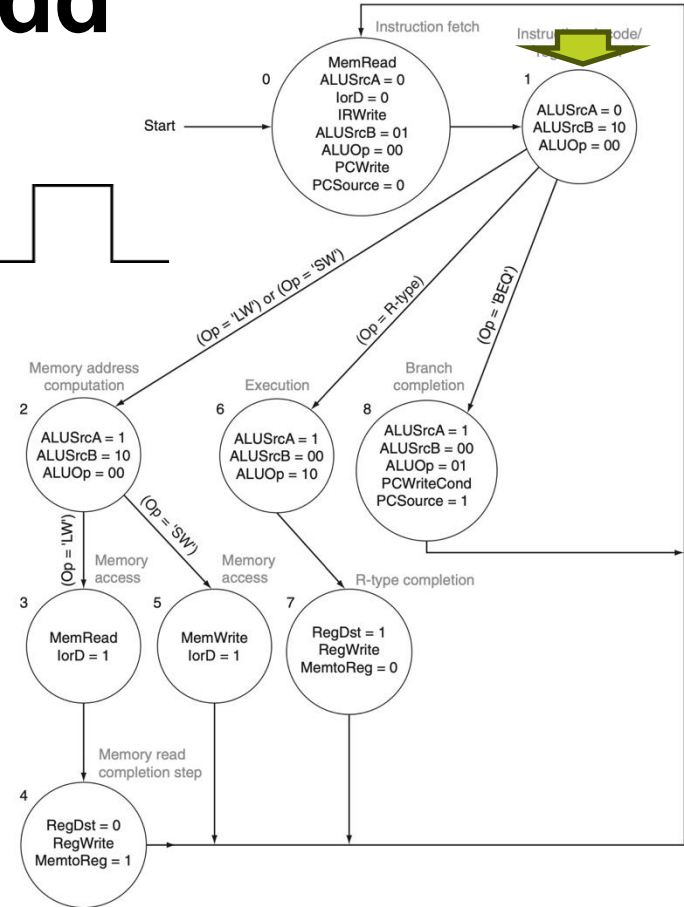
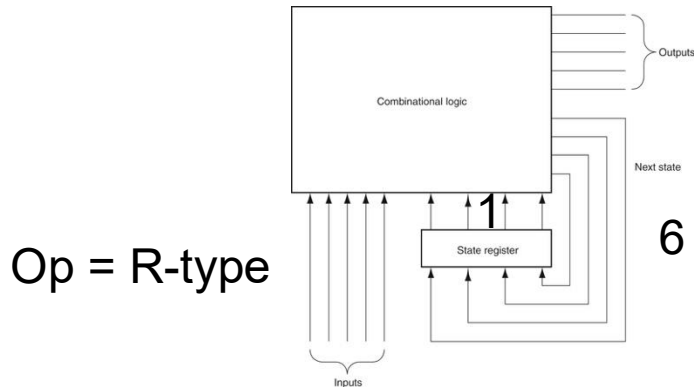
Tilstandsmaskin for Add

Klokke:



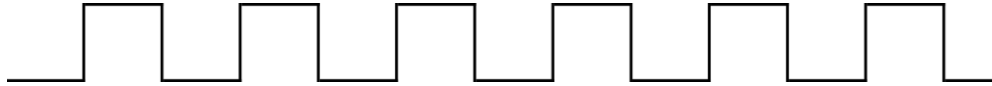
• Instruksjon:

– add x3, x1, x2



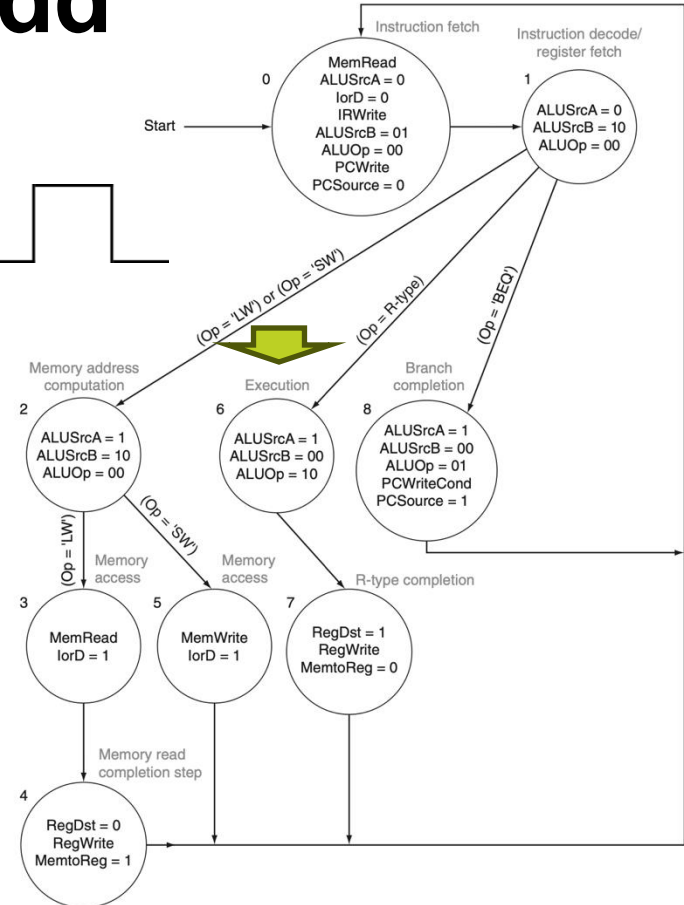
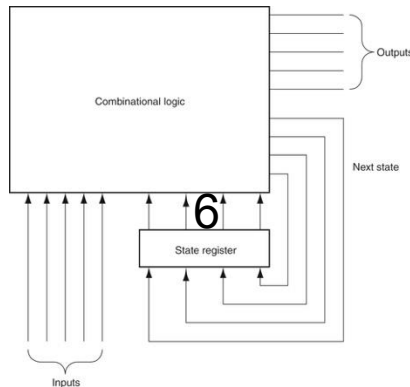
Tilstandsmaskin for Add

Klokke:



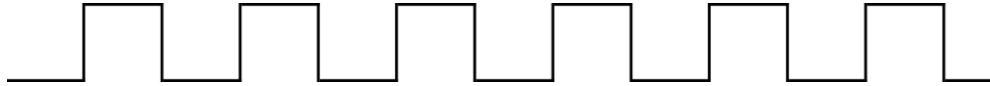
- Instruksjon:
 - add x3, x1, x2

Op = R-type



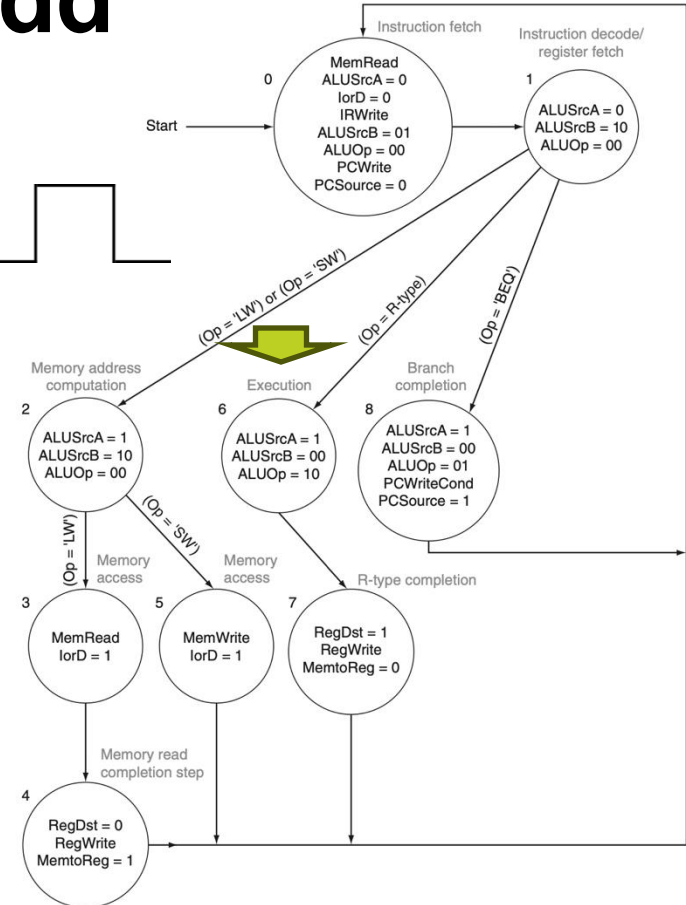
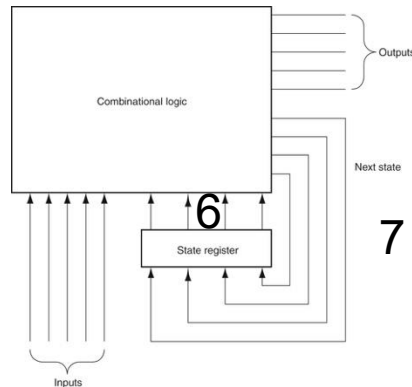
Tilstandsmaskin for Add

Klokke:



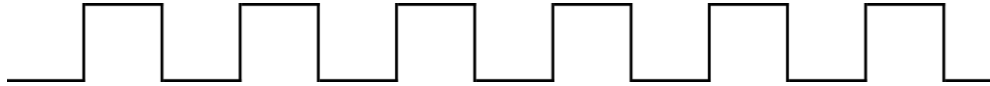
- Instruksjon:
 - add x3, x1, x2

Op = R-type



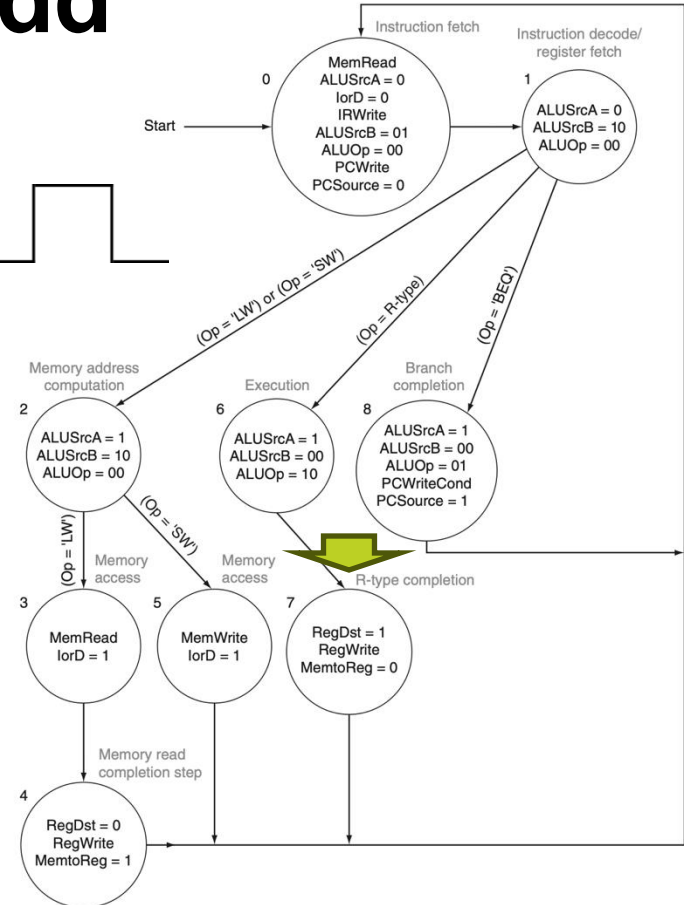
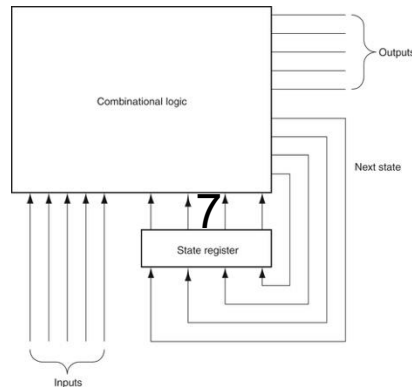
Tilstandsmaskin for Add

Klokke:



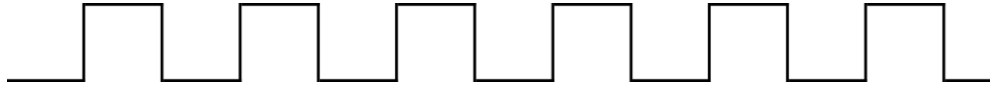
- Instruksjon:
 - add x3, x1, x2

Op = R-type



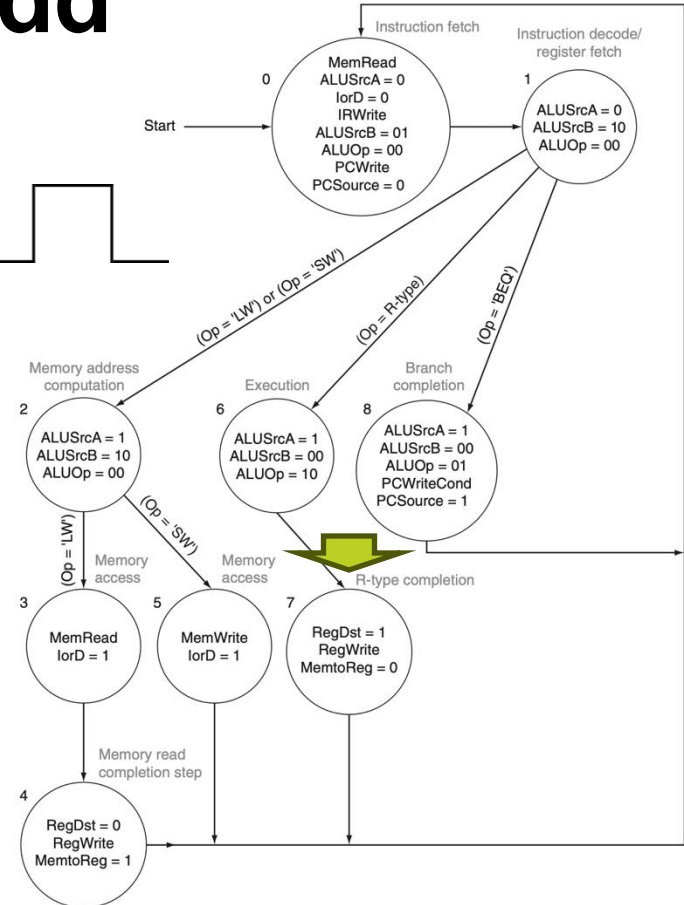
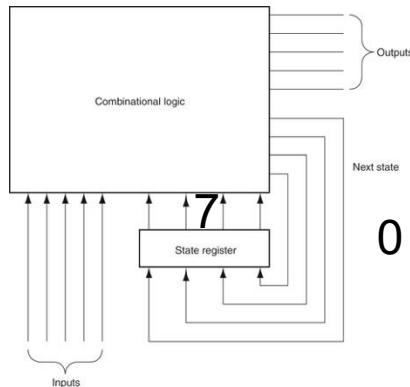
Tilstandsmaskin for Add

Klokke:



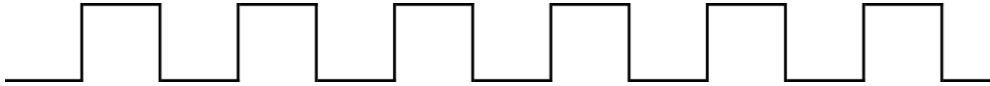
- Instruksjon:
 - add x3, x1, x2

Op = R-type

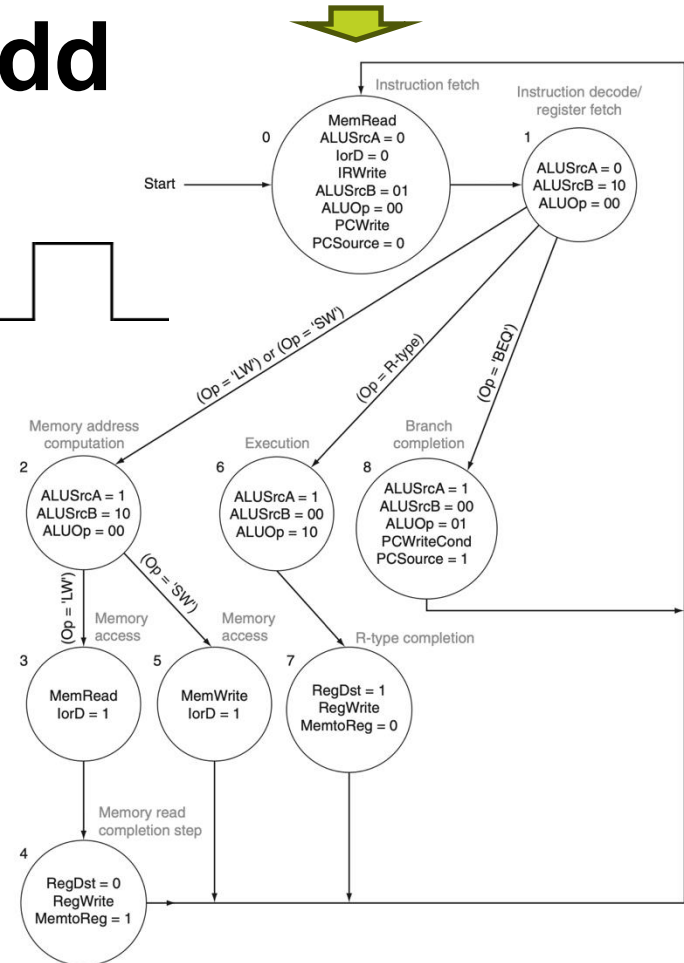


Tilstandsmaskin for Add

Klokke:

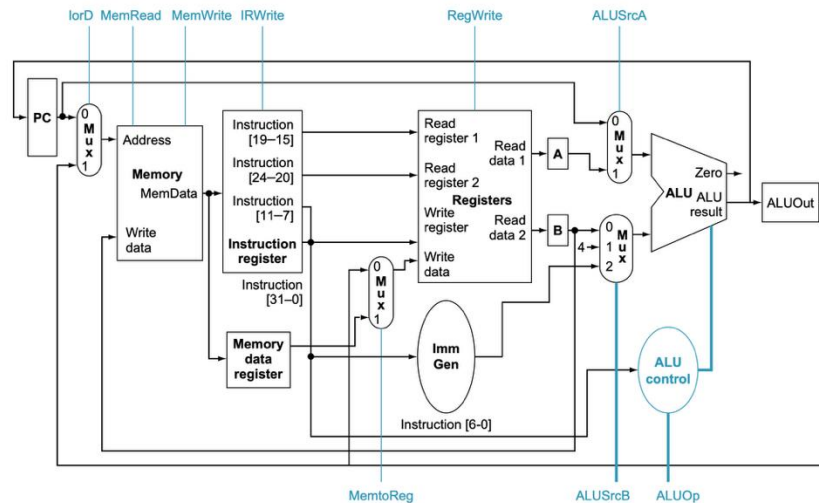


- Instruksjon:
 - `add x3, x1, x2`
- Maskinen er ferdig med add-instruksjonen og henter neste instruksjon



Flersykelprosessorer: Sammendrag

- Flersykelprosessorer er den første arkitekturen vi ser på som faktisk er i bruk
- Grunnen er at de kan bli (veldig) små
 - Små = bruker lite areal på brikken
 - Større areal → høyere produksjonskostnad
- Ytelsen er fortsatt ganske dårlig
 - ... og neste skritt er å se på hvordan vi kan oppnå høyere ytelse

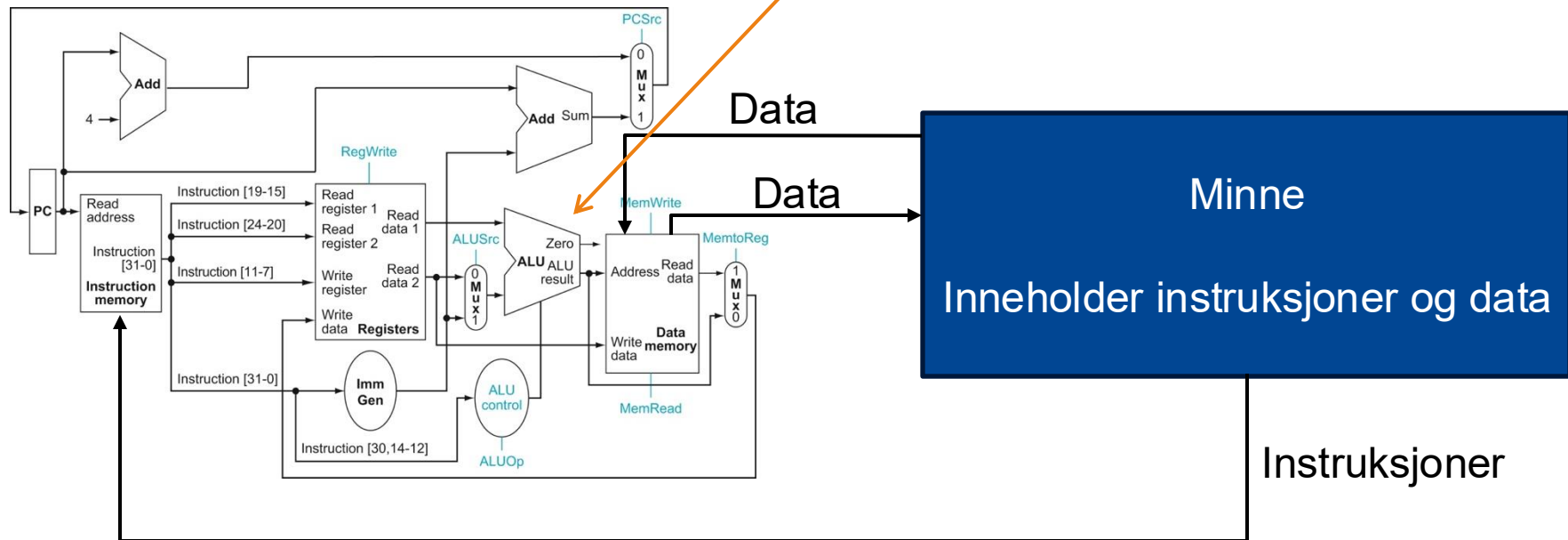




OPPSUMMERING

Prinsippet om lagrede program

Nå vet vi alt vi trenger å vite om denne



... og vi har begynt jobben med å lage raskere og bedre prosessorer